

OBFUSKATION

Entwicklerdokumentation

Aufbau, Bauen und offene Befunde

Inhalt

1. Überblick und Entwurfsentscheidungen

2. Verzeichnisplan und tragende Klassen

Warum die Nutzungsdaten im Index stehen, nicht im Profil

Das kritische Detail beim Umbenennen

3. Datenfluss eines obfuscate-Laufs

4. Das Ableitungsverfahren

5. Schutzmechanismen

6. Erweitern

Neuen Generator hinzufügen

Neue Textregel

Neues Dateiformat

Profilversion anheben

7. Die Oberfläche

8. Bauen, testen, freigeben

9. Konfigurationsschema

10. Rückgabewerte der CLI

Profilverwaltung auf der Kommandozeile

Fassung und Ersteller in der Hilfe

11. Befunde

Fassung 1.1.0 · Stand 7. September 2026

Diese Dokumentation richtet sich an alle, die Obfuskation bauen, erweitern oder abnehmen wollen. Sie setzt Vertrautheit mit C# und .NET voraus und verweist auf konkrete Dateien und Zeilen, statt Code abzuschreiben.

1. Überblick und Entwurfsentscheidungen

Das Projekt besteht aus drei Projekten:

src/Obfuskation.Core/	Klassenbibliothek – die gesamte Fachlogik
src/Obfuskation.Cli/	Kommandozeilenprogramm, eine dünne Hülle darum
src/Obfuskation.Gui/	Oberfläche (Avalonia), ebenfalls nur eine Hülle
tests/	xUnit – Core, Kommandozeile und Oberfläche getrennt

Die Bibliothek kennt weder Konsole noch Fenster: `ObfuscationEngine` gibt nichts aus, wirft stattdessen typisierte Ausnahmen und liefert Berichte als Datenobjekte (`RunReport`).

`Obfuscation.Cli` bildet das auf Konsolenausgabe und Rückgabewerte ab (`ConsoleOutput`, `ExitCodes`), `Obfuscation.Gui` auf Ansichtsmodelle und Fenster.

Warum diese Trennung: dieselbe Engine und dieselbe Ersetzungstabelle für beide Programme bedeutet, dass ein mit der Oberfläche pseudonymisierter Datensatz sich auf der Kommandozeile zurückholen lässt und umgekehrt — ohne diese Zusage wäre das Werkzeug für den Wechsel zwischen interaktiver Arbeit und Skripten unbrauchbar. Abgesichert wird das durch

`tests/Obfuscation.Gui.Tests/EquivalenceTests.cs`: die Tests bauen ein Profil einmal über `ObfuscationEngine` unmittelbar und einmal über `ProfileSession` / `MainViewModel` auf und vergleichen die Ausgabebytes. Am laufenden Programm belegt das Rohprotokoll denselben Befund an echten Dateien (G-08): `diff demo/stammdaten.pseudo.csv`

`arbeit/stammdaten.pseudo.csv` lieferte keine Abweichung — GUI und CLI erzeugten byteweise identische Ausgaben.

2. Verzeichnisplan und tragende Klassen

Klasse / Bereich	Pfad	Aufgabe
ObfuscationEngine	src/Obfuskation.Core/ObfuscationEngine.cs	Einstiegspunkt für Obfuscate, Deobfuscate, Scan und Analyze. Prüft das Profil im Konstruktor und verweigert ein fehlerhaftes sofort.
Profile und Unterklassen	src/Obfuskation.Core/Configuration/Profile.cs	Reines Datenobjekt (Version, Felder, Textregeln, Generatoreinstellungen) ohne Verhalten, damit die Oberfläche es direkt an Formulare binden kann.

Klasse / Bereich	Pfad	Aufgabe
ProfileValidator	src/Obfuskation.Core/Configuration/ProfileValidator.cs	Prüft ein Profil auf Widersprüche und liefert eine Liste von ValidationIssue mit Feldpfad statt einer Sammelmeldung.
MappingStore	src/Obfuskation.Core/Mapping/MappingStore.cs	Verwaltet die Ersetzungstabelle: Laden, Sperren, atomares Schreiben, Rechtevergabe, Rückwärtsindex je Namensraum.
Pseudonymizer	src/Obfuskation.Core/Mapping/Pseudonymizer.cs	Bildet Klartext auf Pseudonym ab und zurück; behandelt Kollisionen und die Sonderfälle nicht umkehrbarer und selbst-umkehrender Generatoren.
SeedDeriver	src/Obfuskation.Core/Generation/SeedDeriver.cs	Leitet deterministische Seeds per HMAC-SHA256 aus dem Profil-Salt ab.
GeneratorRegistry	src/Obfuskation.Core/Generation/GeneratorRegistry.cs	Baut die konfigurierten Generatoren eines Laufs auf, einschließlich eigener Namensräume aus profile.Generators.
TextRuleEngine	src/Obfuskation.Core/Detection/TextRuleEngine.cs	Wendet Textregeln überschneidungsfrei auf Freitext an; alle Regeln laufen gegen den unveränderten Originaltext.
ReverseTextMapper	src/Obfuskation.Core/Detection/ReverseTextMapper.cs	Führt beliebigen Text auf Echtwerte zurück, längster Treffer zuerst, mit Wortgrenzen bei wortartigen Pseudonymen.
GeneratorDescriptions	src/Obfuskation.Core/Generation/GeneratorDescriptions.cs	Deutsche Erklärungen zu jedem eingebauten Generator, gemeinsam genutzt von Oberfläche und Hilfetexten.
Prozessoren	src/Obfuskation.Core/Formats/{Csv,Json,PlainText}Processor.cs	Zerlegen und Zusammensetzen der drei Dateiformate; delegieren die Feldentscheidung an einen IRecordTransformer.

Klasse / Bereich	Pfad	Aufgabe
Transformer	src/Obfuskation.Core/Formats/{Obfuscate, Deobfuscate, Scan}Transformer.cs	Implementieren IRecordTransformer und damit die eigentliche Fachlogik je Vorgang, unabhängig vom Dateiformat.
PathHelper	src/Obfuskation.Core/PathHelper.cs	Pfadauflösung an einer Stelle: ConfigDirectory, ProfileDirectory, DefaultProfilePath, DefaultMappingStorePath und ResolveMappingStore — Letztere gemeinsam genutzt von ObfuscationEngine.ResolveMappingStorePath und ProfileCatalog, damit die Regel nicht zweimal dasteht.
ProfileIndex	src/Obfuskation.Core/Configuration/ProfileIndex.cs	Der Nutzungs-Index: merkt sich Pfade bearbeiteter Datendateien je Profil und wann ein Profil zuletzt benutzt wurde. Getrennt von der Profildatei selbst — Begründung unten.
ProfileCatalog	src/Obfuskation.Core/Configuration/ProfileCatalog.cs	Sammelt alle bekannten Profile (zentraler Ordner plus Zusatzpfade) zu ProfileSummary-Zeilen für Übersicht und profile list; ein unlesbares Profil ergibt einen Eintrag mit gesetztem Error, nie eine Ausnahme.
ProfileRenamer	src/Obfuskation.Core/Configuration/ProfileRenamer.cs	Der sicherheitskritische Teil der Profilverwaltung: benennt ein Profil um, ohne den Bezug zu seiner Ersetzungstabelle zu verlieren (Einzelheiten unten).

Warum die Nutzungsdaten im Index stehen, nicht im Profil

Naheliegender wäre gewesen, Pfad und Zeitpunkt der zuletzt bearbeiteten Dateien gleich im Profil selbst mitzuführen — ein Feld `lastUsedFiles` neben `fields` und `textRules`. Das wurde bewusst nicht so gebaut, aus einem einzigen, aber durchschlagenden Grund: die Profildatei ist die Regeldatei, und die Regeldatei wird **nicht ungefragt geschrieben**. Jede Feldregeländerung in der Oberfläche lebt bis zum ausdrücklichen Klick auf „Speichern“ nur im Speicher (`ProfileSession.HasUnsavedChanges`); genau das ist die Grundlage für die Rückfrage bei ungespeicherten Änderungen (Abschnitt 7). Würde das Öffnen einer Datendatei automatisch das Profil auf der Platte aktualisieren, um den neuen Dateipfad zu vermerken, geschähe damit zwangsläufig eines von zwei unerwünschten Dingen: entweder es würde nur der Nutzungs-Teil geschrieben und der Rest der noch unbestätigten Feldregeln im Speicher ignoriert (eine stille Sonderregel, die schwer nachvollziehbar wäre), oder es würden gleich alle unbestätigten Regeländerungen mitgeschrieben — und genau das darf nicht passieren, weil der Anwender sie noch nicht bestätigt hat. Ein einfaches „Datei geöffnet“ hätte damit den Nebeneffekt, ein halbfertiges Regelwerk festzuschreiben.

Die Trennung in eine eigene, ausschließlich maschinell gepflegte Indexdatei (`profil-index.json`) löst das sauber: `ProfileIndex.RecordDataFile` und `RecordProfileUse` schreiben bei jedem Öffnen unbekümmert und ohne Rückfrage, weil dort ausschließlich Pfade und Zeitstempel stehen — nie eine Feldregel, nie ein unbestätigter Zustand. Der Preis dafür ist, dass die Indexdatei rein informativ ist und beim Löschen (`ProfileIndex.Forget` , „Aus Liste entfernen“ in der Übersicht) nichts an den eigentlichen Profilen oder Ersetzungstabellen ändert — genau das macht sie aber auch gefahrlos löschbar.

Das kritische Detail beim Umbenennen

`ProfileRenamer.Rename` (`src/Obfuskation.Core/Configuration/ProfileRenamer.cs:29`) ändert scheinbar nur ein Textfeld, `Profile.ProfileName` . Der Name bestimmt aber über `PathHelper.DefaultMappingStorePath` auch den *Vorgabepfad* der Ersetzungstabelle, solange `Profile.MappingStore` leer ist — und dieser Vorgabepfad wird beim Anlegen nicht laufend neu berechnet, sondern beim allerersten Lauf einmal aufgelöst und ist von da an implizit an den damaligen Namen gebunden. Würde `Rename` den Namen zuerst ändern, zeigte die Vorgabe für den *neuen* Namen auf ein anderes, leeres Verzeichnis: aus Sicht der nächsten Läufe gäbe es plötzlich keine einzige bekannte Pseudonymisierung mehr, und `Zurückholen` fände nichts. Deshalb schreibt `Rename` als ersten Schritt den bisherigen, aus dem *alten* Namen aufgelösten Pfad ausdrücklich in `Profile.MappingStore` fest (`MappingStorePinned` im `RenameOutcome`), bevor der Name sich ändert. Erst danach folgt der zweite Schritt: existiert die Tabelle bereits, wird der Profilname auch *in ihr* nachgezogen (`MappingStore.RenameProfile`) — ohne diesen Schritt würde `MappingStore.Open` bei jedem folgenden Lauf mit „Der Mapping-Store gehört zum Profil ...“ warnen, weil der im Dokument hinterlegte Name dann vom Profilnamen abweiche. Beide Schritte zusammen sind der Grund, warum Umbenennen überhaupt eigenen Code braucht statt eines einfachen Feldzugriffs; abgesichert durch `ProfileRenameTests` , die wichtigsten Tests des gesamten Vorhabens.

3. Datenfluss eines `obfuscate` -Laufs

1. `ObfuscationEngine` -Konstruktor ruft `ProfileValidator.Validate` auf; bei mindestens einem Befund mit `ValidationSeverity.Error` wirft er sofort `ConfigurationException` — vor jeder weiteren Aktion.
2. `Obfuscate` öffnet den `MappingStore` (`OpenStore`), das legt bei Bedarf Verzeichnis und Datei an, prüft auf ein Git-Arbeitsverzeichnis, setzt die Dateirechte durch und sperrt die Tabelle gegen parallele Läufe.
3. `SeedDeriver` , `GeneratorRegistry` und `Pseudonymizer` werden aus dem Salt des geöffneten Stores aufgebaut.
4. `CreateResolver` baut den `FieldRuleResolver` ; im strengen Modus (`--strict`) arbeitet er auf einer Kopie des Profils mit `UnknownField = FieldAction.Error` , damit das geladene Profil selbst unverändert bleibt.

5. Der passende Prozessor (`CsvProcessor` , `JsonProcessor` oder `PlainTextProcessor`) liest das Byte-Array, erkennt Zeichensatz und bei CSV das Trennzeichen (`TextFormatDetector`), und ruft `transformer.OnFields(...)` mit der vollständigen Feldliste auf.
6. Hier, in `ObfuscateTransformer.OnFields` , erfolgt der Abbruch bei offenen Feldern — vor jeder Verarbeitung eines einzelnen Wertes. Die Methode sammelt zuerst alle unbehandelten Felder und wirft danach in einem Schritt `UnhandledFieldException` mit der vollständigen Liste. Bis dahin wurde kein Byte der Ausgabe geschrieben und kein Eintrag in die Ersetzungstabelle eingetragen.
7. Erst danach verarbeitet der Prozessor Datensatz für Datensatz; je Feld ruft er `transformer.TransformField(...)` auf, das über den `FieldRuleResolver` die passende Regel bestimmt und je nach Aktion durchreicht, schwärzt, entfernt, mit Textregeln durchsucht oder über den `Pseudonymizer` ersetzt.
8. Bei großen Dateien meldet ein `ProgressReporter` den Fortschritt in Blöcken von 500 Datensätzen; ein `CancellationToken` wird je Datensatz geprüft, damit ein Abbruch nicht erst am Ende einer sehr großen Datei wirkt.
9. Ist der Lauf kein Probelauf, schreibt `store.Save()` die Tabelle atomar (erst in eine Nebendatei, dann Umbenennen). Der `RunReport` wird mit Zählern, Feldbehandlungen und Dauer vervollständigt und zurückgegeben.

`Analyze` (für die Voransicht der Oberfläche) durchläuft nur Schritt 1 und liest die Struktur über `FieldInspector` , ohne Werte zu verarbeiten und ohne bei offenen Feldern abubrechen — die Oberfläche muss den offenen Zustand ja gerade anzeigen können.

4. Das Ableitungsverfahren

Jedes Pseudonym wird deterministisch abgeleitet:

```
seed = HMAC-SHA256(Salt des Profils, Generatorname + Klartext + Zähler)
```

(`SeedDeriver.Derive` , `src/Obfuskation.Core/Generation/SeedDeriver.cs:31`). Die Bestandteile werden längenpräfigiert verkettet, damit `("ab", "c")` und `("a", "bc")` nicht denselben Seed ergeben. Aus der Determinismus-Zusage folgt: derselbe Klartext bekommt im selben Profil über alle Dateien und Läufe hinweg dasselbe Pseudonym — das ist es, was Verknüpfungen über Kundennummern intakt hält.

Kollisionsprüfung in beide Richtungen (`Pseudonymizer.Pseudonymize` , `src/Obfuskation.Core/Mapping/Pseudonymizer.cs:57`): beim Eintragen eines neuen Pseudonyms wird geprüft, dass der Kandidat weder bereits als Pseudonym vergeben ist (`IsPseudonymTaken`) noch selbst als Klartext im Bestand steht (`IsPlaintextKnown`) noch mit dem Original identisch ist. Schlägt eine der drei Prüfungen an, wird der Zähler erhöht und neu

abgeleitet, bis zu 100 Versuche (`MaxCollisionRetries`); danach wirft der Pseudonymizer `MappingConflictException` mit dem Hinweis, einen Generator mit größerem Wertevorrat zu wählen.

Warum `dateShift` keinen Tabelleneintrag bekommt: ein verschobenes Datum kann zufällig mit einem echten Datum desselben Bestands zusammenfallen; ein Tabelleneintrag wäre dann mehrdeutig, weil dasselbe Pseudonym-Datum auf zwei verschiedene Klartexte verweisen müsste. `DateShiftGenerator` markiert sich deshalb über `HasIntrinsicInverse => true` (`src/Obfuskation.Core/Generation/DateShiftGenerator.cs:42`), und `Pseudonymizer.Pseudonymize` überspringt für solche Generatoren die Tabelle vollständig — die Rückrechnung erfolgt stattdessen über den profilweit konstanten, aus dem Salt abgeleiteten Offset (`SetOffsetFrom`).

Folge für Freitext: `ReverseTextMapper` (die Rückabbildung in Fließtext) arbeitet ausschließlich über die Einträge der Ersetzungstabelle (`store.AllReverseEntries()`). Da `dateShift` dort keinen Eintrag hinterlässt, kann ein verschobenes Datum, das in freiem Text auftaucht, nicht zurückgeholt werden — nur in CSV- und JSON-Spalten, wo die Feldregel selbst den Bezug zum Generator liefert. Das Rohprotokoll bestätigt das indirekt: bei der Rückübersetzung der KI-Antwort (P-12) traten keine Datumswerte im Freitext auf, die Zurückrechnung betraf dort ausschließlich Namen, Kontonummern, IBAN, BIC, E-Mail, Telefonnummer und Anschrift.

5. Schutzmechanismen

Mechanismus	Umsetzung	Test	Beleg im Rohprotokoll
Rechte 0600 auf der Ersetzungstabelle	<code>FilePermissions.RestrictFile</code> , <code>src/Obfuskation.Core/Mapping/FilePermissions.cs:24</code> ; angewandt in <code>MappingStore.Save</code>	<code>SafetyTests.Die_Ersetzungstabelle_ist_nur_fuer_den_Eigentuer_lesbar</code>	P-07: stat zeigt <code>-rw-----</code>
Verweigerung im Git-Arbeitsverzeichnis	<code>PathHelper.IsInsideGitWorkingTree</code> und die Prüfung in <code>MappingStore.Open</code> , <code>src/Obfuskation.Core/Mapping/MappingStore.cs:71</code>	<code>SafetyTests.Die_Tabelle_verweigert_die_Ablage_in_einem_Git_Verzeichnis</code>	N-05: Rückgabewert 5, keine <code>mapping.json</code> im Repository angelegt
Sperrdatei gegen parallele Läufe	<code>MappingStore.AcquireLock</code> öffnet <code><pfad>.lock</code> mit <code>FileShare.None</code> und <code>DeleteOnClose</code> , <code>src/Obfuskation.Core/Mapping/MappingStore.cs:148</code>	<code>SafetyTests.Ein_zweiter_Lauf_kann_die_Tabelle_nicht_gleichzeitig_beschreiben</code>	N-06: zweiter Lauf nach 0,4 s bricht mit Rückgabewert 5 ab, der erste läuft unbeeinträchtigt zu Ende

Mechanismus	Umsetzung	Test	Beleg im Rohprotokoll
Bericht ohne Werte	RunReport (src/Obfuskation.Core/Reporting/RunReport.cs) führt ausschließlich Zähler, Feld- und Regelnamen; ReportFinding trägt bewusst keinen Fundwert	SafetyTests.Der_Bericht_enthaelt_keinen_einzigen_Eingabewert	P-10: der JSON-Bericht von obfuscat enthält Feldnamen und Zähler, aber keinen einzigen Eingabewert

Ergänzend, ebenfalls in `SafetyTests.cs` festgehalten: der Abbruch bei offenen Feldern erfolgt nachweislich vor jeder Verarbeitung

(`Der_Abbruch_erfolgt_bevor_irgendetwas_verarbeitet_wurde`), ein Probelauf verändert die Tabelle nicht (`Der_Probelauf_veraendert_die_Ersetzungstabelle_nicht`, im Rohprotokoll P-09 mit `trocken.csv`, das nicht angelegt wurde), und eine in sich widersprüchliche Tabelle — dasselbe Pseudonym auf zwei Klartexte — wird beim Laden zurückgewiesen

(`MappingStore.BuildReverseIndex`, `src/Obfuskation.Core/Mapping/MappingStore.cs:174`).

6. Erweitern

Neuen Generator hinzufügen

1. `IPseudonymGenerator` implementieren (`Name`, `IsReversible`, `IsWordLike`, `Generate`; optional `HasIntrinsicInverse` / `TryInvert` und `Configure`) — siehe `src/Obfuskation.Core/Generation/SimpleGenerators.cs` für einfache Beispiele.
2. Die Instanz in `GeneratorRegistry.CreateDefaults()` (`src/Obfuskation.Core/Generation/GeneratorRegistry.cs:21`) eintragen.
3. **Pflicht:** einen Eintrag in `GeneratorDescriptions` (`src/Obfuskation.Core/Generation/GeneratorDescriptions.cs:13`) hinzufügen. Ohne ihn schlägt `GeneratorDescriptionTests.Jeder_eingebaute_Generator_hat_eine_Erklärung` fehl — der Test zählt bewusst über `GeneratorRegistry.KnownNames`, damit ein neuer Generator ohne deutsche Erklärung nicht unbemerkt durchrutscht und in der Oberfläche als nacktes englisches Wort erscheint.
4. Bei Bedarf `ValidateGenerators` in `ProfileValidator.cs` erweitern, falls der neue Generator eigene Pflichteinstellungen mitbringt.

Neue Textregel

Textregeln sind Profildaten (`TextRule`), keine eigenen Klassen. Ein neues Standardmuster gehört in `ProfileScaffolder.DefaultTextRules()`

(`src/Obfuskation.Core/Configuration/ProfileScaffolder.cs:139`), mit Priorität, Muster, Ziel-

Generator und — falls ein Präfix wie `IBAN:` stehen bleiben soll — `CaptureGroup`. Ein zu weit gefasstes Muster fällt in der Oberfläche im Erprobungsfeld auf (`TextRulesViewModel.Evaluate`); ein Test dafür liegt in `DefaultTextRuleTests.cs`.

Neues Dateiformat

Ein neuer Prozessor implementiert keine eigene Schnittstelle für die Fachlogik — er ruft stattdessen dieselben drei Transformer-Typen (`ObfuscateTransformer`, `DeobfuscateTransformer`, `ScanTransformer`) über `IRecordTransformer` auf (`src/Obfuskation.Core/Formats/IRecordTransformer.cs`). Zu implementieren sind: Struktur einlesen und `OnFields` mit der vollständigen Feldliste aufrufen (bevor irgendetwas geschrieben wird), `ShouldDrop` vor dem Schreiben jedes Feldes befragen, `TransformField` je Wert aufrufen und das Ergebnis übernehmen. `ObfuscationEngine.ResolveFormat` und `RunProcessor` (`src/Obfuskation.Core/ObfuscationEngine.cs:302`) müssen den neuen Fall kennen, ebenso `FieldInspector.Inspect` für die Strukturvorschau ohne Verarbeitung.

Profilversion anheben

`Profile.CurrentVersion` und `MappingDocument.CurrentVersion` (`src/Obfuskation.Core/Configuration/Profile.cs:9` bzw. `src/Obfuskation.Core/Mapping/MappingDocument.cs:15`) sind getrennte Zähler. `ProfileValidator.Validate` verweigert ein Profil mit höherer Version als Fehler, `MappingStore.Open` ebenso einen Store mit höherer Version als `MappingConflictException`. Ein Versionssprung sollte immer mit einer nachvollziehbaren Migration oder zumindest einer klaren Fehlermeldung einhergehen, welche Version das Programm mindestens braucht.

7. Die Oberfläche

MVVM ohne Framework: `ObservableObject` und `RelayCommand` / `AsyncRelayCommand` (`src/Obfuskation.Gui/ViewModels/ObservableObject.cs`, `RelayCommand.cs`) sind Eigenbau, keine externe Bibliothek. `MainViewModel` kennt weder `Window` noch einen Dateialog unmittelbar:

- **Dialoge hinter einer Schnittstelle:** der Konstruktor von `MainViewModel` nimmt `Func<IDialogService> dialogs` entgegen (`src/Obfuskation.Gui/Services/IDialogService.cs`); erst beim tatsächlichen Aufruf entsteht der echte `DialogService` mit Bezug auf das laufende Fenster (`src/Obfuskation.Gui/Services/DialogService.cs`), die einzige Umsetzung der Schnittstelle. Dadurch lässt sich das Ansichtsmodell in Tests mit einer Fabrik füttern, die entweder eine Ausnahme wirft, falls doch ein Dialog aufgerufen würde (`tests/Obfuskation.Gui.Tests/EquivalenceTests.cs:125`), oder — für die Profilverwaltung — mit `FakeDialogService` (`tests/Obfuskation.Gui.Tests/FakeDialogService.cs`) vorgegebene Antworten liefert: welche Wahl bei der Rückfrage „ungespeicherte

Änderungen“ getroffen wird, welcher Name und Ablageort beim Anlegen herauskommen, oder welches Profil aus der Übersicht gewählt wird — alles ohne ein einziges echtes Fenster.

- **Nebenfenster als Ereignis** (die drei bestehenden aus Fassung 1.0.0, dazu `HelpRequested` für die Kurzhilfe aus 1.1.0) und **Rückfragen über `IDialogService`** (neu in 1.1.0):
`TextRulesRequested`, `MappingRequested`, `AboutRequested` und `HelpRequested` sind einfache `Action`-Ereignisse (`src/Obfuskation.Gui/ViewModels/MainViewModel.cs:78`); `MainWindow.axaml.cs` abonniert sie und öffnet das jeweilige Fenster. Die Kurzhilfe (`HelpWindow.axaml`) folgt damit demselben schlichten Muster wie die drei älteren Nebenfenster, nicht dem `IDialogService` der Profilverwaltung — sie liefert kein Ergebnis, auf das der Aufrufer warten müsste, sondern zeigt nur an. Die drei neuen Fenster der Profilverwaltung — `NewProfileWindow`, `RenameProfileWindow`, `ProfilesWindow` (alle unter `src/Obfuskation.Gui/Views/`) — laufen dagegen über `IDialogService`-Methoden, die ein `Task<TErgebnis?>` liefern: `AskNewProfileAsync`, `AskRenameProfileAsync`, `ShowProfilesAsync`. Der Unterschied zu den älteren Nebenfenstern: diese drei brauchen ein Ergebnis, auf das der Aufrufer wartet (den gewählten Namen, das gewählte Profil), ein Ereignis ohne Rückgabewert würde dafür nicht reichen. `ConfirmWindow` (`src/Obfuskation.Gui/Views/ConfirmWindow.axaml`) ist der schlichte, wiederverwendbare Meldungsdialog mit bis zu drei Schaltflächen dahinter — Avalonia bringt keinen mit; er bedient sowohl `AskSaveChangesAsync` als auch die Umbenennen-Rückfrage. Jede Schaltfläche schließt das Fenster mit ihrem `Tag` als Ergebniswert; ohne Auswahl (etwa Schließen über die Titelleiste) liefert `ShowDialog<string?>` `null`, was der Aufrufer als die vorsichtige Richtung wertet — „Abbrechen“, nie „Verwerfen“.
- **Warum es die Kurzhilfe gibt:** `docs/anwenderdokumentation.md` erklärt das Programm vollständig, aber eine Anleitung, die neben dem Programm liegt, wird erfahrungsgemäß nicht gelesen — wer die Oberfläche zum ersten Mal öffnet, öffnet kein zweites Dokument dazu. Ohne einen Hinweis unmittelbar im Programm erschließt sich insbesondere nicht, wozu ein Profil überhaupt gut ist: dass es Feldregeln und Ersetzungstabelle bündelt und deshalb dieselbe Datei bei einem falschen Profil ein anderes Pseudonym bekäme. Die Kurzhilfe (`HelpWindow.axaml`) trägt diese Erklärung deshalb in eine einzige, bewusst kurz gehaltene Ansicht im Programm selbst, mit einem Verweis am Ende auf die ausführliche Anwenderdokumentation für alles, was darüber hinausgeht.
- **Schutz vor Datenverlust:** `MainViewModel.EnsureChangesHandledAsync()` (`src/Obfuskation.Gui/ViewModels/MainViewModel.cs`) fragt über `AskSaveChangesAsync` nach, sobald `HasUnsavedChanges` zutrifft, und liefert `false`, wenn der Anwender „Abbrechen“ wählt; aufgerufen vor `LoadProfile` in `OpenProfileAsync`, `NewProfileAsync` und dem Öffnen eines Profils aus der Übersicht (`ShowProfilesAsync`). Beim Fensterschließen greift dieselbe Methode über einen kleinen Umweg: `Window.Closing` kann nicht auf eine `Task` warten, deshalb bricht `App.axaml.cs` den ersten Schließversuch mit `e.Cancel = true` ab, startet `EnsureChangesHandledAsync()` und ruft bei `true` über einen Merker `_closeConfirmed` ein zweites Mal `window.Close()` — der Merker muss vor diesem

zweiten Aufruf gesetzt sein, sonst entstünde eine Schleife aus Abbrechen und erneutem Rückfragen.

- **Themen** liegen unter `src/Obfuskation.Gui/Themes/` (`Colors.axaml` , `Controls.axaml` , `Metrics.axaml`); `ThemeService.Apply` (`src/Obfuskation.Gui/Services/ThemeService.cs`) setzt nur `Application.Current.RequestedThemeVariant` , die Farbumschaltung selbst übernimmt Avalonias `ThemeDictionaries` .

Weil die Bedienlogik so von der laufenden Anwendung entkoppelt ist, lässt sie sich ohne Fenster prüfen — `tests/Obfuskation.Gui.Tests/MainViewModelTests.cs` tut das für Profilwahl, Feldregeln und die drei Vorgänge, sowie neu für die Rückfrage bei ungespeicherten Änderungen und das Anlegen eines Profils; `tests/Obfuskation.Gui.Tests/ProfilesViewModelTests.cs` prüft Sortierung, Filter und „Aus Liste entfernen“ der Übersicht, ebenfalls ohne ein echtes Fenster.

`MappingSummary` (`src/Obfuskation.Gui/Services/MappingSummary.cs`) zieht die Logik, die bislang nur `MappingViewModel` kannte — Pfad, Anzahl der Einträge, Existenz der Tabelle —, in eine kleine, eigenständige Hilfsklasse, die jetzt sowohl `MappingViewModel` als auch die neue Untertitelzeile der Kopfzeile (`MainViewModel.ProfileSubtitle`) verwenden. Neu berechnet wird sie nur beim Profilladen und nach einem abgeschlossenen Lauf, nicht bei jeder einzelnen Feldregeländerung — die Tabelle ändert sich schließlich auch nur bei einem tatsächlichen Lauf.

8. Bauen, testen, freigeben

```
dotnet build
dotnet test
```

FISH

Zielframework ist `net8.0` mit `RollForward=Major` in `Directory.Build.props` — auf einem Rechner ohne installierte .NET-8-Laufzeit laufen `dotnet run` und `dotnet test` dadurch trotzdem auf der vorhandenen neueren Laufzeit, während das Zielframework `net8.0` bleibt.

Freigabe über `build-release.sh` (Windows-Gegenstück: `build-release.cmd` , gleiche Optionen):

```
./build-release.sh                # win-x64 UND linux-x64, je beide Programme
./build-release.sh --rid linux-x64 # nur diese Laufzeit
./build-release.sh --cli-only      # ohne Oberfläche
./build-release.sh --gui-only      # nur die Oberfläche
./build-release.sh --self-contained # ohne installiertes .NET lauffähig
./build-release.sh --no-single-file # nicht zu einer Datei zusammenfassen
./build-release.sh --output <pfad> # abweichendes Wurzelverzeichnis
```

FISH

Das Skript veröffentlicht `Obfuskation.Cli` und `Obfuskation.Gui` einzeln statt über die Solution, damit Bibliothek und Testprojekt nicht mitgezogen werden. Es setzt `-p:DebugType=none` (keine `.pdb` -Dateien im Auslieferungs- verzeichnis) sowie `-p:PublishSingleFile=true` und `-`

`p:IncludeNativeLibrariesForSelfExtract=true`, damit auch die nativen Bibliotheken der Oberfläche (Skia, HarfBuzz) in der einen Programmdatei landen. Ohne `--self-contained` erwartet das Ergebnis eine installierte .NET-8-Runtime; mit `--self-contained` bringt es sie mit, auf Kosten der Dateigröße. Das Ergebnis liegt unter `publish/<RID>/` und besteht je Laufzeit aus zwei Dateien: `obfuskation` und `obfuskation-gui`.

9. Konfigurationsschema

Aus `src/Obfuskation.Core/Configuration/Profile.cs` und `Enums.cs`.

Profile

Feld	Typ	Vorgabe	Wirkung
version	int	1	Muss $\leq$ der vom Programm unterstützten Version sein, sonst Fehler
profileName	string	"default"	Name des Profils; bestimmt den Standardpfad der Ersetzungstabelle und ist im Store hinterlegt
description	string?	null	Freitext des Anwenders, wofür das Profil da ist. Rein erklärend, ohne Wirkung auf die Verarbeitung; wird von der Oberfläche in Kopfzeile und Profilübersicht angezeigt. Optionales Feld seit Fassung 1.1.0 — <code>Profile.Version</code> bleibt trotzdem 1, da <code>System.Text.Json</code> unbekannte Felder überliest und alte Programme neue Profile ohne diese Angabe lesen können
mappingStore	string?	null	Pfad zur Mapping-Datei, ~ wird aufgelöst; leer heißt <code>~/.local/share/obfuskation/<profileName>/mapping.json</code>
input	InputSettings	siehe unten	Einstellungen zum Einlesen
defaults	ProfileDefaults	siehe unten	Vorgaben für Felder ohne eigene Regel
fields	List<FieldRule>	leer	Feldregeln; die erste passende gewinnt
textRules	List<TextRule>	leer	Muster für Freitext und unstrukturierte Dateien

Feld	Typ	Vorgabe	Wirkung
generators	Dictionary<string, GeneratorSettings>	leer	Generatoreinstellungen und eigene Namensräume, adressiert über den Schlüssel

InputSettings

Feld	Typ	Vorgabe	Wirkung
csvDelimiter	string?	null	null = aus der Kopfzeile erkennen; sonst erzwungen
encoding	string?	null	null = erkennen (BOM, dann strenges UTF-8, sonst Windows-1252); sonst erzwungen
hasHeaderRecord	bool	true	Ob die erste CSV-Zeile Spaltennamen enthält; ohne Kopfzeile werden Spalten über ihre Position benannt

ProfileDefaults

Feld	Typ	Vorgabe	Wirkung
unknownField	FieldAction	Error	Behandlung eines Feldes ohne eigene Regel; Passthrough erzeugt eine Warnung bei der Profilprüfung
redactionPlaceholder	string	"***"	Platzhalter für Redact

FieldRule

Feld	Typ	Vorgabe	Wirkung
match	string	""	Muster für den Feldnamen gemäß matchType; darf nicht leer sein
matchType	FieldMatchType	Exact	Exact (Groß-/Kleinschreibung egal), Regex oder JsonPath (nur JSON)
action	FieldAction	Error	Die Behandlung des Feldes
generator	string?	null	Generatorname, Pflicht bei Pseudonymize
textRules	List<string>?	null	Namen der bei ScanText angewandten Textregeln; leer/null heißt alle

Feld	Typ	Vorgabe	Wirkung
comment	string?	null	Freitext für den Menschen, wird von init gesetzt

TextRule

Feld	Typ	Vorgabe	Wirkung
name	string	""	Eindeutiger Name, muss gesetzt und einmalig sein
priority	int	50	Höhere Werte gewinnen bei überlappenden Treffern
pattern	string	""	Regulärer Ausdruck, muss gültig sein
generator	string	"token"	Generator für Treffer dieser Regel
captureGroup	int	0	Gruppennummer, deren Inhalt ersetzt wird; 0 = gesamter Treffer
ignoreCase	bool	false	Groß-/Kleinschreibung beim Musterabgleich ignorieren

GeneratorSettings

Feld	Typ	Vorgabe	Wirkung
type	string?	null	Zugrundeliegender eingebauter Generator; leer heißt: wie der Schlüssel selbst — ein anderer Wert erzeugt einen eigenen Namensraum auf Basis dieses Typs
maxDays	int	400	Maximaler Betrag der Datumsverschiebung in Tagen (nur dateShift)
formats	List<string>?	null	Zusätzlich erkannte Datumsformate (nur dateShift), vor den eingebauten Formaten geprüft
country	string?	null	Ländercode für iban/bic, falls sich keiner aus dem Originalwert ableiten lässt
domain	string?	null	Domain für email; bei redact wird dieses Feld zweckentfremdet als Platzhaltertext verwendet

10. Rückgabewerte der CLI

Aus `src/Obfuskation.Cli/ExitCodes.cs`. Sie sind Teil der Schnittstelle: Skripte werten sie aus, sie ändern sich nicht stillschweigend.

Wert	Bedeutung
0	Erfolg

Wert	Bedeutung
1	allgemeiner Fehler
2	Konfiguration fehlt oder ist fehlerhaft
3	Feld ohne Regel im strengen Modus
4	scan hat Verdachtsfälle gefunden
5	Ersetzungstabelle widersprüchlich oder gesperrt

Profilverwaltung auf der Kommandozeile

`CommandContext.LoadProfile` (`src/Obfuskation.Cli/CommandContext.cs:18`) nimmt für `--config` seit Fassung 1.1.0 auch einen bloßen Profilnamen an, nicht mehr nur einen Dateipfad: enthält der übergebene Wert kein Verzeichnistrennzeichen und endet nicht auf `.json`, und existiert er nicht wörtlich als Datei, wird zusätzlich unter `PathHelper.DefaultProfilePath` nachgesehen. Ein Pfad mit Trennzeichen oder mit `.json`-Endung wird weiterhin ausschließlich wörtlich genommen — diese Unterscheidung verhindert, dass eine tatsächlich vorhandene, aber (noch) nicht auffindbare Datei stillschweigend gegen ein gleichnamiges zentrales Profil getauscht wird. Schlägt beides fehl, nennt die Fehlermeldung beide versuchten Orte und verweist auf `obfuskation profile list`. Abgesichert durch `tests/Obfuskation.Cli.Tests/CommandContextTests.cs` — dem ersten Testprojekt für die Kommandozeile, mit derselben `TestUmgebung`-Umleitung über `XD_CONFIG_HOME` / `XD_DATA_HOME` wie in den beiden anderen Testprojekten (Befund D-8).

`obfuskation profile list [--sort name|used|changed] [--json] (Program.cs, BuildProfileCommand)` speist sich aus `ProfileCatalog.Collect(ProfileIndex.Load(), [])` — bewusst ohne Zusatzpfade: die Zuletzt-Liste der Oberfläche (`gui.json`) kennt die Kommandozeile nicht, nur den plattformübergreifenden Nutzungs-Index. Die Ausgabe läuft über zwei neue Methoden in `ConsoleOutput`: `WriteProfiles` (Tabelle auf die Standardfehlerausgabe, wie `WriteSummary`) und `WriteProfilesJson` (auf die Standardausgabe). Sortiert wird in `Program.cs` selbst, nicht in `ProfileCatalog` — dieselbe Aufgabenteilung wie zwischen `ProfileCatalog` und `ProfilesViewModel` auf der Oberflächenseite.

`obfuskation init` kennt zwei neue Optionen: `--description <text>` setzt `Profile.Description`, `--central` schreibt nach `PathHelper.DefaultProfilePath(profileName)` statt nach `obfuskation.json` im aktuellen Verzeichnis — eine ausdrücklich angegebene `--config` gewinnt in jedem Fall. Die Ausgabe nennt seither immer den vollständigen, aufgelösten Zielpfad statt eines möglicherweise relativen.

Fassung und Ersteller in der Hilfe

Unter jeder Hilfeausgabe des Kommandozeilenprogramms — der des Programms selbst wie der jedes Unterbefehls — steht eine einzelne Zeile der Form

```
obfuskation 1.1.0 · Gregor Stübner & Claude (Anthropic)
```

Sie kommt aus `src/Obfuskation.Cli/ProgramInfo.cs`. Die Fassung stammt aus `AssemblyInformationalVersionAttribute`, also aus `<Version>` in `Directory.Build.props`; der angehängte Commit wird für die Anzeige abgeschnitten. Da `System.CommandLine` keinen Platz für eigenen Text unter der Hilfe vorsieht, umschließt `FooterHelpAction` die vorhandene Hilfeaktion, statt sie zu ersetzen — so bleibt die erzeugte Hilfe unverändert und die Zeile kommt nur hinten dran. Die Oberfläche zeigt dieselbe Angabe im Fenster „Über“ (`src/Obfuskation.Gui/Views/AboutWindow.axaml`).

11. Befunde

Die folgenden neun Befunde stammen aus der Testdurchführung vom 4. und 5. September 2026 (`docs/bilder/protokoll-roh.md`). Keiner betrifft die Richtigkeit der Ersetzung, die Umkehrbarkeit oder den Schutz der Ersetzungstabelle.

Zwei davon sind in Fassung 1.0.1 behoben — D-4 und D-8. Sie stehen hier weiterhin, weil eine Befundliste, aus der Behobenes verschwindet, ihren Wert als Nachweis verliert: sie zeigt dann nicht mehr, was geprüft wurde. Was geändert wurde, steht bei den beiden Einträgen und ausführlich in `docs/testdokumentation.md`, Abschnitt „Behebung“.

Nr.	Art	Stand
D-1	Fehler, kosmetisch	offen
D-2	Eigenschaft	offen, dokumentiert
D-3	Bedienbarkeit	offen
D-4	Fehler, zerstörend	behoben in 1.0.1
D-5	Darstellung	offen
D-6	Darstellung	offen
D-7	Sprache	offen
D-8	Testhygiene	behoben in 1.0.1
D-9	Fehler, Auslieferung	offen

D-1 (Fehler, kosmetisch) — Doppelte Befundliste bei fehlerhaftem Profil. Ursache:

`ObfuscationEngine` setzt die Befunde bereits in den Meldungstext der `ConfigurationException` (`src/Obfuskation.Core/ObfuscationEngine.cs:86`), und `CommandContext.Run` hängt sie anschließend noch einmal einzeln an (`src/Obfuskation.Cli/CommandContext.cs:103`). Keine Auswirkung auf Ergebnis oder Rückgabewert, die Ausgabe ist nur doppelt so lang wie nötig (belegt in N-03). Vorschlag: in `CommandContext.Run` beim Fangen von `ConfigurationException` nur die Kopfzeile der Meldung ausgeben, nicht den vollständigen `Message` -Text, der die Befunde bereits enthält.

D-2 (Eigenschaft, dokumentationspflichtig) — `deobfuscate` mit falschem Profil liefert bei Datumsspalten stillschweigend falsche Werte. Ursache: `dateShift` besitzt keinen

Tabelleneintrag (Abschnitt 4) und rechnet stattdessen über den profilweiten Offset zurück; mit einem falschen Profil ist dieser Offset ein anderer, das Ergebnis ein plausibles, aber falsches Datum, während alle anderen Spalten korrekt als `unbekanntesPseudonym` gemeldet werden (N-04, Rückgabewert bleibt 0). Vorschlag: keine Codeänderung nötig, da mit dem Prinzip aus Abschnitt 4 unvermeidlich — stattdessen in der Anwenderdokumentation und im Bericht selbst deutlicher machen, dass die Befundzahl bei richtigem Profil 0 sein muss (außer bei `redact` / `drop`); denkbar wäre ein eigener Warnhinweis im Bericht, sobald `dateShift` -Treffer neben mindestens einem `unbekanntesPseudonym` -Befund auftreten.

D-3 (Bedienbarkeit) — Oberfläche verweist bei fehlerhafter Konfiguration auf nicht sichtbare Hinweise. Ursache: der Hinweisbereich hängt am ausgewählten Feld

(`MainViewModel.RefreshIssues` befüllt `Issues` , aber die Anzeige liegt im Regelbereich, der ohne wählbares Feld leer bleibt); bei fehlerhafter Konfiguration bleibt die Feldliste leer, weil sich keine Engine aufbauen lässt (`src/Obfuskation.Gui/ViewModels/MainViewModel.cs:414`). Belegt in G-20: die Statuszeile meldet nur „Die Konfiguration ist fehlerhaft — siehe Hinweise“, während die Kommandozeile dieselben fünf Fehler mit Feldpfad nennt (N-03). Vorschlag: die `Issues` -Liste zusätzlich unabhängig vom gewählten Feld anzeigen, etwa in einem eigenen Bereich oberhalb der Feldliste, solange keine Engine aufgebaut werden kann.

D-4 (Fehler, zerstörend) — Generatorauswahl blieb leer bei einem eigenen Namensraum und löschte den Generator. Behoben in 1.0.1. Ursache: `GeneratorOption.Find` suchte nur in

`BuiltIn` und legte sonst ein neues `GeneratorOption(name, "eigener Namensraum")` an; die Auswahlliste stammt aber aus `GeneratorOption.For` , wo derselbe Eintrag die Erklärung „eigener Namensraum, wie numericId“ trägt. `GeneratorOption` ist ein `record` und vergleicht über den Wert — die beiden Instanzen waren also nicht gleich, die ComboBox fand keinen passenden Eintrag und setzte ihren `SelectedItem` auf `null` . Der Setter von `SelectedGenerator` schrieb dieses `null` in die Regel zurück: **das bloße Ansehen des Feldes entfernte den Generator.** Wurde das Profil danach gespeichert, war er dauerhaft weg; wer ihn von Hand ergänzte und dabei `numericId` statt `beLegNummer` wählte, hob die Trennung der Namensräume auf. Dieselbe Ursache traf `TextRuleViewModel.Generator` .

Behebung: `Find` nimmt jetzt das Profil entgegen und sucht in `For(profile)`; `TextRuleViewModel` bekommt das Profil gereicht. Festgehalten durch `Ein_Feld_mit_eigenem_Namensraum_zeigt_seinen_Generator_an` und `Auch_eine_Textregel_zeigt_einen_eigenen_Namensraum_an`, die beide prüfen, dass der gewählte Eintrag in der Auswahlliste enthalten ist — genau daran scheiterte es.

D-5 (Darstellung) — Bildlaufleiste überdeckt die Anzahlen im Fenster „Ersetzungstabelle“. Die rechtsbündigen Zahlen je Namensraum werden teilweise angeschnitten. Betroffen ist vermutlich das Layout in `src/Obfuskation.Gui/Views/MappingWindow.axaml` — der Container für die Liste müsste der Bildlaufleiste Platz reservieren (etwa über einen rechten Rand oder `Padding` auf dem Inhalt statt auf dem `ScrollView`).

D-6 (Darstellung) — Hinweistext im Fenster „Textregeln“ rechts abgeschnitten. Der Text „höhere Priorität gewinnt bei Überlappung“ ist unvollständig lesbar. Betroffen ist `src/Obfuskation.Gui/Views/TextRulesWindow.axaml`; Abhilfe wäre Zeilenumbruch (`TextWrapping="Wrap"`) statt fester Breite für dieses Element.

D-7 (Sprache) — `--help` mischt Deutsch und Englisch. `Description:` und „Show help and usage information“ stammen aus den Vorgaben von `System.CommandLine` (`src/Obfuskation.Cli/Program.cs`) und wurden nicht lokalisiert. Kosmetisch, keine Auswirkung auf die Bedienung. Vorschlag: prüfen, ob `System.CommandLine` einen Weg bietet, diese Textbausteine zu überschreiben; sonst als bekannte Einschränkung dokumentieren.

D-8 (Testhygiene) — `dotnet test` überschrieb die echte `~/.config/obfuskation/gui.json` des Benutzers. Behoben in 1.0.1. Beobachtet: vor dem Testlauf verwies `recentProfiles` auf ein Wegwerfverzeichnis, danach auf ein anderes. Ursache: `MainViewModelTests` legt zwar ein Wegwerfverzeichnis für Profile an, erzeugte die Einstellungen aber mit `new GuiSettings()`; `MainViewModel` ruft an vier Stellen `_settings.Save()` auf, und `GuiSettings.FilePath` (`src/Obfuskation.Gui/Services/GuiSettings.cs:47`) löst statisch auf `$XDG_CONFIG_HOME/obfuskation/gui.json` auf — ohne gesetzte Variable also auf das echte Benutzerverzeichnis. Keine Datenschutzgefahr, da die Datei nur Pfade und die gewählte Ansicht speichert, keine Werte aus verarbeiteten Dateien; ein Testlauf muss trotzdem rückwirkungsfrei sein.

Behebung: `tests/Obfuskation.Gui.Tests/TestUmgebung.cs` setzt über einen `[ModuleInitializer]` `XDG_CONFIG_HOME` auf ein Wegwerfverzeichnis, bevor der erste Test angefasst wird, und räumt es beim Beenden weg. Der Modulinitialisierer gilt für alle Testklassen der Baugruppe — anders als eine Änderung im Konstruktor einer einzelnen Klasse. Festgehalten durch `Der_Testlauf_fasst_die_Einstellungen_des_Anwenders_nicht_an`.

D-9 (Fehler, Auslieferung) — „Speichern“ schlägt in der veröffentlichten Oberfläche fehl. Der Klick auf `Speichern` bricht mit `Could not load file or assembly 'System.IO.Pipelines, Version=9.0.0.0' ab`, das Profil wird nicht geschrieben. Eingegrenzt: der nicht gebündelte Build (`dotnet .../obfuskation-gui.dll`) speichert fehlerfrei, das Kommandozeilenprogramm ebenso,

und `ScanAndConfigTests.Profile_ueberstehen_das_Schreiben_und_Lesen_unveraendert` besteht. Der Fehler steckt also nicht in `ProfileStore`, sondern in der Auslieferung als Einzeldatei: das Bündel enthält `System.IO.Pipelines` 8.0.23 (über CsvHelper hereingezogen), verdeckt damit die Fassung der Laufzeit, und unter `RollForward=Major` auf einer neueren Laufzeit verlangt deren `System.Text.Json` die Assemblyfassung 9.0.0.0. Auf einem Rechner mit der in der README geforderten .NET-8-Laufzeit tritt der Fehler nicht auf. Vorschlag: entweder die .NET-8-Laufzeit als Voraussetzung durchsetzen und beim Start prüfen, oder mit `--self-contained` ausliefern, oder das Zielframework auf die tatsächlich ausgelieferte Laufzeit heben — die Entscheidung gehört zur Auslieferung, nicht zum Code.

Erstellt von Gregor Stübner und Claude (Anthropic).